MEI Secure Software

Practical Project

Academic Year 2025–2026

1 2 9 0

UNIVERSIDADE D
COIMBRA

2026-03-16 — v1.3: updated final delivery description to include example of exploit
2026-03-12 — v1.2: updated admin description (should also be able to enable users); add clarification that removing all the previous codebase is not allowed
2026-03-10 — v1.1: updated weight distribution; defined technical stack; removed checkpoint
2026-03-10 — Project assignment v1 released

This project aims to expose you to the **end-to-end process of designing, implementing, and validating security controls across the software development lifecycle**.

You will start from an **existing application and CI/CD workflow that were not designed with security in mind**. The goal is to systematically introduce security considerations from the early phases of development (requirements and architecture) and propagate them through implementation and automated DevSecOps controls.

The project therefore bridges three major aspects of secure software engineering:

1. **Security Requirements Engineering**
2. **Secure Architecture and Design**
3. **Secure Implementation and DevSecOps Integration**

It is expected that you apply established methodologies and concepts from the literature, including:

- **SQUARE – Security Quality Requirements Engineering**
- **Threat Modeling**
- **Misuse Cases**
- **Attack Trees**
- **Defense in Depth**
- **DevSecOps security integration**

2. Context

You are part of a development team that has **inherited a partially developed** web service and its associated CI/CD pipeline.

The system currently provides several basic functionalities, but:

- security requirements were **never formally defined**
- the architecture was **not designed with explicit security principles**
- the implementation may contain **vulnerabilities or incorrect countermeasures**
- the CI/CD workflow **does not integrate security analysis or controls**

Your task is to **transform this system into a secure service supported by a secure delivery pipeline**. You are **not allowed to discard the existing codebase and rebuild the application from scratch**. Instead, you must **analyze the current implementation, identify the existing security issues, and fix them within the provided codebase**. While modifications and refactoring are expected, the original structure of the system must be preserved and incrementally improved.

This project adopts a **security-by-design** approach. Students must derive security requirements from threat analysis and implement architectural and technical controls that enforce those requirements throughout the entire development lifecycle.

2.1. Target System

The system considered in this project is a **web-based document management platform** that allows users to upload, store, and manage digital documents. Uploaded documents may contain sensitive or private information.

The platform is intended to support multiple types of users, each with different permissions and responsibilities. Typical users of the system include:

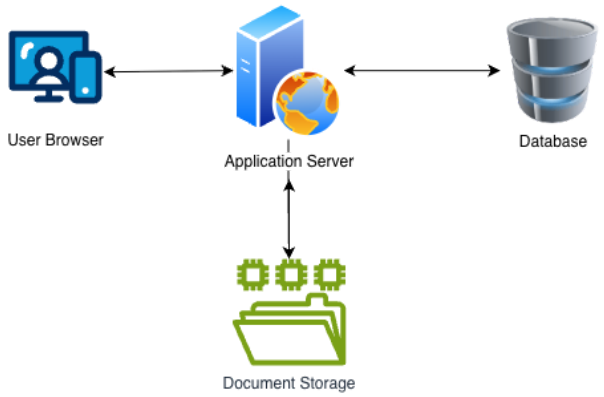
- **Users**, who can upload and manage their own documents
- **Reviewers**, who are users that have been granted access to documents shared with them for review or processing
- **Administrators**, who manage the platform and oversee system operations

The provided system includes a minimal implementation of the document management platform. The baseline provides the basic infrastructure required to run and extend the system, including:

- a simple login page with basic authentication
- document upload functionality
- basic document listing functionality
- database persistence using PostgreSQL
- a CI/CD workflow skeleton for building and testing the application

The application is implemented as a web service accessible through a browser-based client. A backend service handles application logic and interacts with a database and file storage system.

A simplified architectural view of the system is shown below.



2.2. Functional Scope

For the purposes of this project, assume that the system must support the following functionality.

User

A user must be able to:

- authenticate to the platform
- upload a document
- list documents they own
- download documents they own
- share a document with another user for review

Reviewer

A reviewer is a user who has been granted access to documents owned by other users. A reviewer must be able to:

- list documents shared with them
- download documents shared with them

Administrator

An administrator must be able to:

- list registered users
- disable/enable a user account

2.3. Application Baseline

Students will be provided with an existing incomplete implementation of the system. The system is containerized, and is implemented in Python. Javascript, HTML, CSS, and SQL are also used.

The codebase provides the some of functionality described above, but may contain:

- incomplete functionalities
- implementation vulnerabilities
- incorrect or incomplete security countermeasures
- insecure configuration
- insecure dependency usage

Students are expected to analyze the existing implementation and introduce the necessary improvements to enforce the security requirements identified during the project.

2.4. CI/CD Baseline

Students will also receive a working CI/CD workflow that supports:

- building the application
- running basic tests
- packaging artifacts

However, this pipeline was designed primarily for functionality and **does not currently integrate security analysis or security controls**.

Students must extend this workflow to incorporate **DevSecOps practices**, integrating automated security checks and verification steps into the pipeline.

3. Project Structure

The project is structured into several stages that progressively bridge **requirements, architecture, and implementation**.

3.1. Stage 1 — Security Requirements Engineering

Following the **SQUARE methodology**, define the security requirements for the system.

Focus on the following SQUARE steps:

- Step 2 – Identify security goals
- Step 3 – Develop artifacts (threat modeling, misuse cases, attack trees)
- Step 4 – Perform risk assessment
- Step 6 – Elicit security requirements
- Step 7 – Categorize and prioritize requirements

The analysis should include:

Assets and Stakeholders

Identify relevant stakeholders and assets that must be protected.

Threat Modeling

Perform a structured threat modeling analysis of the system. This analysis should identify potential threats, attack vectors, and system abuse scenarios that could compromise the security of the system.

Misuse Case Analysis

Develop misuse case diagrams or scenarios describing how the system could be abused by an attacker.

Attack Trees

Develop attack trees for selected critical misuse cases.

Security Requirements

Derive security requirements from the identified threats.

Security requirements should be:

- clear and unambiguous
- testable and verifiable
- traceable to the security goals identified for the system
- justified by the threats or misuse cases identified during the threat modeling step

3.2. Stage 2 — Secure Architecture and Design

Based on the previously defined security requirements, propose architectural mechanisms and design decisions that enforce those requirements.

Rather than describing the full system architecture, this stage should focus on **security-relevant architectural decisions**. The goal is to explain **how the system should be structured to enforce the security requirements and mitigate the identified threats**.

Students should clearly explain how each architectural decision addresses the threats identified in Stage 1 and supports the defined security requirements.

A simple diagram may be included to illustrate the placement of security mechanisms and trust boundaries. The diagram should emphasize **security responsibilities and trust relationships**, rather than providing a complete system architecture description.

Architectural decisions should be traceable to the security requirements derived in Stage 1.

Trust Boundaries and Security Domains

Identify the main **trust boundaries** in the system and describe the security domains they separate.
Discuss potential attack surfaces that arise at these boundaries.

Security Enforcement Points

Describe where and how key security mechanisms are enforced in the architecture. Explain how these enforcement points mitigate the threats identified in Stage 1.

Architectural Security Principles

Discuss how the proposed design applies established security principles, such as:

- **least privilege**
- **defense in depth**
- **secure defaults**
- **separation of concerns**

Students should explain how these principles influence the architectural decisions made.

3.3. Stage 3 — Secure Implementation

Using the provided baseline application, students must implement or improve security controls according to the security requirements and architectural decisions defined in the previous stages.

The objective of this stage is to translate the proposed security architecture into concrete implementation mechanisms within the application.

Students should analyze the baseline codebase and introduce the necessary modifications to enforce the identified security requirements. This may include correcting vulnerabilities, improving existing mechanisms, or introducing new security controls where necessary.

All implementation decisions should be clearly justified and linked to the corresponding security requirements and architectural decisions.

3.4. Stage 4 — DevSecOps Integration

Security mechanisms introduced in the previous stages should not rely solely on manual verification. In modern software development processes, security checks are integrated into the development workflow to enable **continuous security verification** and early detection of vulnerabilities.

This approach is commonly referred to as **DevSecOps**, where security practices are embedded throughout the development pipeline. By integrating security checks into the CI/CD workflow, security issues can be detected earlier in the development lifecycle (a principle often referred to as **shift-left security**).

Students must extend the provided CI/CD workflow to include automated security analysis (e.g., SAST, DAST, ...) and verification steps. The goal of this stage is to ensure that security requirements and architectural decisions defined earlier in the project can be **continuously verified during development and integration**.

Students should explain:

- which security checks were integrated
- how they relate to the security requirements identified in Stage 1
- how the pipeline enforces the security policies defined in the project
- evidence of pipeline and security tools execution (e.g., git logs, resources, screenshots, ...)

3.5. Stage 5 — Security Validation (Optional / Bonus)

As an advanced step, teams may deploy their hardened system on a dedicated virtual machine exposed to the network.

The deployment allows instructors to perform an independent security validation of the system, such as:

- the effectiveness of implemented security controls
- the presence of remaining vulnerabilities

- the robustness of the deployment configuration

4. Milestones

The project will be developed throughout the semester with two main submission milestones.

Intermediate Submission — 30%

Deadline: ~~March 27~~ April 3 (postponed)
Defense: ~~Week 8~~ Week 10 (Week of April 13) (postponed)

Teams must submit a **preliminary version of the project report** describing the work completed so far. The submission should include:

- a description of the **system scope and assets**
- the **threat analysis**, including misuse cases and attack trees
- the defined **security goals and security requirements**
- the proposed **secure architecture and design**, including identification of trust boundaries and security mechanisms

The intermediate submission should demonstrate that the **security requirements and architectural design are sufficiently developed to guide the implementation phase of the project**.

Final Submission — 70%

Deadline: May 15
Defenses: Week 15 (Week of May 18)

Teams must submit the **final version of the project report** describing the completed system and the work conducted throughout the project. The submission should include:

- **the url to the project repository (shared with users *jrcampos* and *joseadp*)**
- a description of the **final system architecture and design**
- the **final version of the security goals and security requirements**
- the implemented **security mechanisms and controls**
- the **implementation changes and extensions** made to the baseline application
- the **mitigation of identified vulnerabilities or insecure design choices**
- **one or two example exploits demonstrating vulnerabilities that exist in the baseline system** (if any)
- the **integration of security verification mechanisms in the CI/CD pipeline**
- evidence of **pipeline execution and security analysis results**

- a discussion of the **traceability between threats, requirements, architecture, implementation, and verification mechanisms**

The final submission should demonstrate that the **implemented system enforces the defined security requirements and integrates security verification into the development lifecycle**.

Important!

Enrollment in defenses mandatory at Inforestudante (slots will be made available):

- every student must participate, defenses are individual
- each student must demonstrate a comprehensive understanding of the project as a whole (approaches, design, conception) regardless of task division within the group
- learning is not intended to be segmented, and all students must be able to explain and justify all major aspects of the work

5. Assessment

The project will be evaluated based on the quality of the analysis, design decisions, implementation, and integration of security practices throughout the development lifecycle.

The project is to be done in groups of **3-4.**

1. Security Requirements Engineering — 20% (Int: 15% - Final: 5%)

Evaluation criteria include:

- identification of relevant assets and stakeholders
- quality of threat modeling artifacts (misuse cases, attack trees)
- clarity and completeness of security goals
- quality and testability of the derived security requirements
- traceability between threats and requirements

2. Secure Architecture and Design — 20% (Int: 15% - Final: 5%)

Evaluation criteria include:

- identification of trust boundaries and attack surfaces
- justification of architectural security decisions
- placement of security enforcement points
- application of architectural security principles (e.g., least privilege, defense in depth)
- traceability between security requirements and architectural mechanisms

3. Secure Implementation — 30% (Final: 30%)

Evaluation criteria include:

- correctness and effectiveness of implemented security controls
- mitigation of vulnerabilities or insecure mechanisms in the baseline system
- alignment between implementation changes and architectural decisions
- quality and clarity of code modifications

4. DevSecOps Integration — (Final: 25%)

Evaluation criteria include:

- integration of security verification mechanisms into the CI/CD pipeline
- relevance of the selected checks with respect to the security requirements
- correctness and completeness of pipeline configuration
- evidence of pipeline execution and generated security reports

5. Report Quality and Traceability — (Final: 5%)

Evaluation criteria include:

- clarity and organization of the report
- quality of explanations and justification of decisions
- explicit traceability between:
  threats → security requirements → architectural decisions → implementation → pipeline verification

6. Optional Bonus — Security Validation — (Final: 10%)

Teams that deploy their system and participate in the optional validation stage may receive **bonus points** based on:

- robustness of the deployed system and effectiveness of implemented controls during validation **(5%)**
- improvement based on the feedback/outcome of the validation phase **(5%)**